



Wydział Matematyki i Nauk Informatycznych

POLITECHNIKA WARSZAWSKA

Metody Sztucznej Inteligencji 2

Raport końcowy

*Wykorzystanie i usprawnienie MCTS do implementacji sztucznej
inteligencji grającej w Connect4*

Filip Filipkowski
Antoni Kingston

Streszczenie

Raport przedstawia implementację i empiryczną ocenę agentów grających w zmodyfikowany wariant *Connect4* typu *suicide* na planszy 6×8 z ruchami **drop** i **push**. Zaimplementowano silnik gry, graczy opartych o minimax, Monte Carlo Tree Search w wariantach UCT, FPU, LGR i PMBp (tryb online, 1000 iteracji na ruch), gracza losowego oraz interfejs do modeli językowych. Przeprowadzono turniej round-robin dziewięciu agentów (360 partii), wstępną ocenę jakości ruchów metryką *Blunder Rate* względem przybliżonej wyroczni UCT oraz 70 partii człowiek–komputer z pięcioma uczestnikami.

W turnieju najwyższy *Win Rate* osiągnął PMBp (76%), przed UCT (65%) i minimaxem $d=4$ (60%); modele językowe zajęły ostatnie miejsca (6–11%). PMBp wygrał bezpośrednie pojedynki z pozostałymi wariantami MCTS, co potwierdza hipotezę o skuteczności co najmniej jednej modyfikacji UCT; FPU i LGR nie poprawiły wyniku względem UCT. *Blunder Rate* odróżnił słabych graczy od MCTS, lecz przy obecnej wyroczni (UCT, 20 000 iteracji) nie dostarczył danych wystarczających do rozstrzygnięcia hipotezy o odporności PMBp na pułapki taktyczne. W testach z człowiekiem (70 partii, pięciu uczestników) ludzie wygrali 28 partii, przegrali 31 i zremisowali 11; przeciwko wariantom MCTS — 7 zwycięstw, 26 porażek i 7 remisów na 40 gier. Uczestnicy wygrali wszystkie partie z graczem losowym oraz osiem z dziesięciu z minimaxem $d=3$. Przewaga pierwszego gracza nie została wyeliminowana w sensie przyjętym w hipotezie (45–55% *Win Rate* jako czerwony). Ograniczenia obejmują liczbę partii na parę, przybliżony charakter wyroczni, brak treningu self-play przed turniejem oraz wąski zakres testów ludzkich i LLM.

Słowa kluczowe: Monte Carlo Tree Search, UCT, Connect4, wariant suicide, First Play Urgency, Last Good Reply, Power-Mean Backpropagation, Blunder Rate

Spis treści

Streszczenie	1
1 Opis problemu	3
1.1 Cel projektu	3
1.2 Monte Carlo Tree Search i warianty UCT	3
1.2.1 UCT	3
1.2.2 FPU	3
1.2.3 LGR	3
1.2.4 PMBp	4
1.3 Badana modyfikacja gry Connect4	4
1.3.1 Wersja standardowa	4
1.3.2 Wariant badany w projekcie	4
2 Implementacja aplikacji	5
2.1 Architektura modułów	5
2.2 Silnik gry	5
2.3 Agenci i przeszukiwanie MCTS	5
2.3.1 Gracze porównawczy	5
2.3.2 MCTS w trybie online	6
2.3.3 Gracz LLM	7
2.4 Interfejsy użytkownika	7
2.5 Pipeline eksperymentalny	7

3	Sprawdzone hipotezy i opis eksperymentów	7
3.1	Postawione hipotezy	7
3.2	Porównywani gracze	8
3.3	Metodyka eksperymentalna	8
3.3.1	Turniej automatyczny round-robin	8
3.3.2	Wyrocznia i metryka Blunder Rate	9
3.3.3	Porównanie z modelami językowymi	9
3.4	Testy człowiek–komputer	9
3.5	Metryki oceny	10
4	Wyniki eksperymentów	11
4.1	Wyniki turnieju agentów	11
4.2	Porównanie wariantów MCTS	14
4.3	Blunder Rate i jakość gry	16
4.4	Testy z udziałem człowieka i modeli językowych	17
4.4.1	Testy człowiek–komputer	17
4.4.2	Porównanie między modelami językowymi	18
4.5	Obserwacje dodatkowe	19
5	Wnioski	20
5.1	Weryfikacja hipotez	20
5.2	Podsumowanie	21
6	Deklaracja użycia narzędzi sztucznej inteligencji	22
6.1	Uwaga końcowa	23

1 Opis problemu

Celem projektu jest empiryczne porównanie metod sztucznej inteligencji w grze złożonej taktycznie: zmodyfikowanym wariantcie *Connect4* typu *suicide*. Badamy, czy i w jakim stopniu Monte Carlo Tree Search w wariantcie UCT oraz jego modyfikacje (FPU, LGR, PMBp) przewyższają prostsze podejścia oraz—w szerszym ujęciu—jak wypadają względem człowieka i modeli językowych.

1.1 Cel projektu

Klasyczna gra *Connect4* na planszy 6×7 została rozwiązana: pierwszy gracz może wymusić zwycięstwo przy optymalnej grze [1]. Dlatego w projekcie przyjmujemy bardziej złożony wariant *suicide*, w którym celem jest uniknięcie utworzenia własnej linii czterech żetonów, a plansza ma wymiary 6×8 . Dodatkowy ruch typu **push** oraz nietypowe kryterium końca partii zwiększają przestrzeń stanów i utrudniają stosowanie prostych heurystyk znanych z klasycznej wersji gry [8]. Głównym pytaniem badawczym jest ocena skuteczności wariantów MCTS w takim środowisku oraz identyfikacja modyfikacji algorytmu, które realnie poprawiają jakość gry.

1.2 Monte Carlo Tree Search i warianty UCT

Monte Carlo Tree Search to metoda przeszukiwania drzewa decyzyjnego oparta na losowych symulacjach (rolloutach). W przeciwieństwie do minimaxa z ręcznie zaprojektowaną funkcją oceny, MCTS uczy się wartości pozycji na podstawie wyników wielu rozegranych partii próbnych [2]. Pojedyncza iteracja składa się z czterech faz: **wyboru** ścieżki od korzenia do liścia, **ekspansji** drzewa o nowy węzeł, **symulacji** losowej gry do stanu końcowego oraz **propagacji wstecznej** wyniku do odwiedzonych węzłów.

1.2.1 UCT

Podstawowym wariantem stosowanym w projekcie jest UCT (*Upper Confidence Bound applied to Trees*) [6]. W fazie wyboru preferowany jest ruch a maksymalizujący:

$$\text{UCT}(s, a) = \bar{X}_{s,a} + C \sqrt{\frac{\ln N(s)}{N(s, a)}},$$

gdzie $\bar{X}_{s,a}$ to średni wynik symulacji po ruchu a ze stanu s , $N(s)$ i $N(s, a)$ oznaczają liczbę odwiedzin stanu i liczbę wykonań ruchu, a C reguluje równowagę między eksploatacją (pierwszy składnik) a eksploracją (drugi składnik).

1.2.2 FPU

Modyfikacja *First Play Urgency* dotyczy traktowania jeszcze nieodwiedzonych ruchów w fazie wyboru. Zamiast pomijać je do momentu pierwszej ekspansji, otrzymują one zadaną wartość początkową (parametr FPU), co pozwala kontrolować, jak agresywnie algorytm eksploruje nowe gałęzie drzewa [3]. Zbyt wysoka wartość FPU sprzyja płytkiemu przeszukiwaniu, zbyt niska — przedwczesnej koncentracji na początkowo obiecujących liniach.

1.2.3 LGR

Last Good Reply zmienia fazę symulacji: algorytm zapamiętuje ruchy, które w danej pozycji prowadziły do korzystnego wyniku, i preferuje je podczas kolejnych rolloutów [5]. Celem jest ograniczenie szumu wynikającego z w pełni losowych symulacji bez wprowadzania pełnej heurystyki pozycyjnej.

1.2.4 PMBp

W standardowym MCTS wartość węzła to średnia arytmetyczna wyników symulacji. W wariancie *Power-Mean Backpropagation* stosuje się średnią potęgową [4]:

$$V_p = \left(\frac{1}{k} \sum_{i=1}^k a_i^p \right)^{\frac{1}{p}},$$

gdzie a_i to wyniki pojedynczych symulacji, a parametr p steruje wpływem wartości skrajnych. Dla $p = 1$ otrzymujemy UCT w klasycznym wariancie backpropagation; im większe p , tym silniej ważymy zwycięstwa, im niższe tym silniej porażki.

1.3 Badana modyfikacja gry Connect4

1.3.1 Wersja standardowa

Connect4 to gra dwuosobowa, deterministyczna i z pełną informacją. W klasycznym wariancie na planszy 6×7 gracze na przemian wrzucają żetony do kolumn; żetony opadają na najniższe wolne pole. Wygrywa ten, kto pierwszy ułoży cztery własne żetony w jednej linii (poziomej, pionowej lub ukośnej). Plansze o innych proporcjach, np. 8×6 , pojawiają się także w literaturze dotyczącej ogólnego programowania gier [7].

1.3.2 Wariant badany w projekcie

W projekcie przyjmujemy wariant *Don't Connect 4* (suicide). Poniżej zestawiono jego reguły.

Plansza i ruchy. Gra toczy się na planszy 6×8 . W każdej turze gracz wykonuje jeden z dwóch typów ruchu w wybranej niepełnej kolumnie:

- **drop** — żeton spada na najniższe wolne pole kolumny (jak w klasycznym *Connect4*),
- **push** — żeton wpychany jest od spodu kolumny, a dotychczasowe żetony w tej kolumnie przesuwają się o jedno pole w górę.

Ruch **push** pozwala m.in. niszczyć istniejące układy na planszy, co znacząco zwiększa liczbę legalnych kontynuacji w porównaniu z samym **drop**.

Cel gry i liczenie linii. Gracz dąży do *uniknięcia* utworzenia własnej linii czterech żetonów.

Zakończenie partii. Obowiązuje reguła **natychmiastowej porażki właściciela linii**: jeśli w danym ruchu powstaje nowy segment czterech żetonów, przegrywa gracz, do którego należy ten segment—niezależnie od tego, kto wykonał ruch (własną linię lub wymuszenie przeciwnikowi). Jeżeli w jednym ruchu powstają linie obu graczy, partia kończy się **remisem**. Jeżeli w ruchu nie powstaje żadna nowa linia, gra trwa dalej aż do zapełnienia planszy, które skutkuje **remisem**.

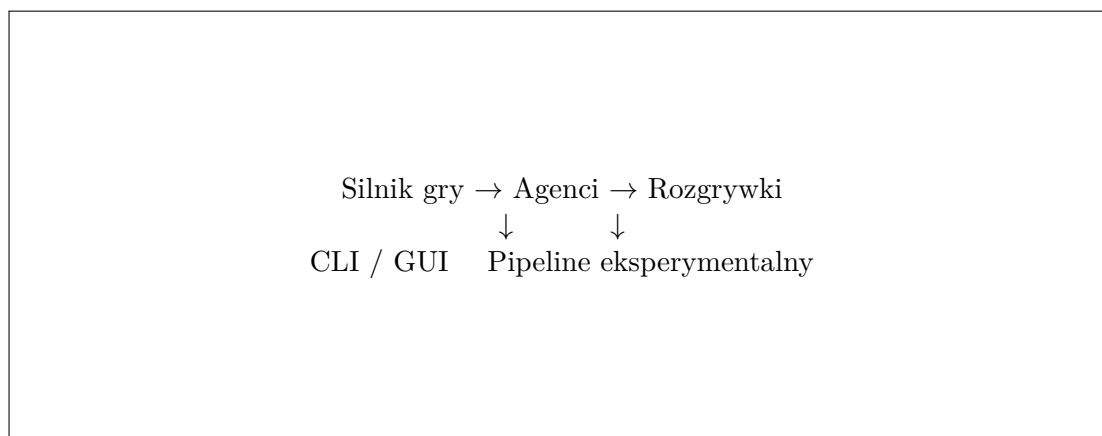
Motywacja modyfikacji. Połączenie wariantu suicide, ruchu **push** i planszy 6×8 tworzy przestrzeń stanów trudniejszą do opanowania heurystyką typową dla klasycznego *Connect4* (gdzie premiuje się własne zagrożenia ofensywne). Parzysta liczba kolumn łagodzi przewagę gracza rozpoczynającego wynikającą z zajęcia środka w wersji 6×7 . Złożoność taktyczna wariantu—krótkoterminowe pułapki, wymuszanie linii przeciwnikowi, konieczność planowania skutków **push**—stanowi wymagające środowisko testowe dla algorytmów MCTS i ich modyfikacji.

2 Implementacja aplikacji

Aplikacja została zaimplementowana w języku Python 3.11+ jako pakiet `connect4_mcts` wraz ze skryptami pomocniczymi. Kod podzielono na warstwę logiki gry, moduł agentów, mechanizm rozgrywek oraz pipeline eksperymentalny. Projekt objęto testami automatycznymi weryfikującymi poprawność zasad gry, zachowanie agentów i spójność wyników turniejów.

2.1 Architektura modułów

Główne komponenty systemu przedstawia rysunek 1. Rdzeń stanowi silnik gry z niezmiennym modelem stanu (`GameState`), typem ruchu (`Move`) oraz logiką legalności i końca partii. Warstwa agentów implementuje wspólny protokół (`choose_move`) dla gracza losowego, minimaxa, wariantów MCTS oraz modelu językowego. Moduł rozgrywek przeprowadza partie agent-agent, a interfejsy CLI i GUI udostępniają grę człowiekowi. Eksperymenty wsadowe obejmują turniej, ocenę *Blunder Rate* oraz pełny pipeline eksperymentalny konfigurowany deklaratywnie (format TOML).



Rysunek 1: Uproszczony przepływ zależności między modułami aplikacji.

2.2 Silnik gry

Stan gry reprezentowany jest jako niezmienny obiekt `GameState` zawierający planszę 6×8 , gracza na ruchu oraz status partii. Metoda `apply_move` została mocno zoptymalizowana, ze względu na stanowienie zdecydowanej większości obciążenia obliczeniowego: legalność ruchu sprawdzana jest w czasie $O(1)$, przebudowywane są wyłącznie zmienione komórki planszy, a nowe segmenty czterech żetonów wykrywane są inkrementalnie na podstawie różnicy stanów przed i po ruchu (zamiast pełnego skanowania planszy).

2.3 Agenci i przeszukiwanie MCTS

Wszyscy gracze implementują ten sam interfejs `Agent` z metodą `choose_move(state)`. Fabryka agentów buduje graczy na podstawie nazwy typu (`random`, `minimax`, `uct`, `fpu`, `lgr`, `pmbp`, `llm`) oraz hiperparametrów przekazywanych z CLI, GUI lub konfiguracji eksperymentu.

2.3.1 Gracze porównawczy

Random wybiera jednostajnie spośród legalnych ruchów `drop` i `push`.

Minimax z przycinaniem α - β . Gracz **Minimax** przeszukuje drzewo ruchów do zadanej głębokości d (parametr konfigurowalny; w turnieju **minimax-d3** ma $d = 3$, **minimax-d4** ma $d = 4$). W każdym węźle wybierany jest ruch maksymalizujący wartość heurystyczną z perspektywy gracza korzenia; na poziomach przeciwnika wartość jest minimalizowana. Stosowane jest przycinanie α - β . Gdy $d = 0$ lub brak legalnych kontynuacji, zwracana jest ocena liścia.

Funkcja oceny pozycji. Liście oraz pozycje nie rozstrzygnięte w ramach budżetu głębokości oceniane są dedykowaną funkcją oceny pozycji. Wynik podawany jest z *perspektywy gracza, dla którego w danym momencie wykonywane jest przeszukiwanie* (gracza korzenia przy wyborze ruchu).

Dla stanu **terminalnego** (partia zakończona):

- zwycięstwo ocenianego gracza: $+10^6$,
- porażka: -10^6 ,
- remis: 0.

Wartość terminalna zależy wyłącznie od zwycięzcy, a nie od liczby linii w wyniku końcowym.

Dla stanu **nieterminalnego**:

$$H(s, p) = P(s, p_{\text{opp}}) - P(s, p), \quad (1)$$

gdzie p to oceniany gracz, p_{opp} — jego przeciwnik, a $P(s, \cdot)$ — *potencjał linii* danego koloru w pozycji s . Jest to odwrócenie klasycznej heurystyki *Connect4*: w wariacie *suicide* korzystna jest pozycja, w której **przeciwnik** ma duży potencjał zbudowania linii (można go do niej zmusić), a **własny** potencjał jest mały (unikamy tworzenia własnych zagrożeń).

Potencjał linii $P(s, c)$. Dla koloru c funkcja sumuje wkłady wszystkich *okien czterech pól* na planszy 6×8 : każde okno to cztery sąsiednie komórki w jednym z czterech kierunków (poziomo, pionowo, po obu przekątnych). Dla okna w niech n_c i n_{opp} oznaczają liczbę żetonów odpowiednio koloru c i przeciwnika w w . Jeśli $n_{\text{opp}} > 0$, okno jest **pomijane** (linia tego koloru w tym miejscu jest zablokowana). W przeciwnym razie do sumy dodawana jest waga zależna od n_c :

n_c (żetony c w oknie)	waga
1	1
2	3
3	9
0 lub 4	0

Wagi rosną wykładniczo z liczbą własnych żetonów w otwartym oknie — trzy żetony w jednym rzędzie bez blokady przeciwnika dają wkład 9, co silnie karze zbliżanie się do własnej linii. Okna zawierające żeton przeciwnika nie wliczają się do potencjału danego koloru, co odzwierciedla niemożność dokończenia linii przez zablokowane układy.

2.3.2 MCTS w trybie online

Klasa **MCTSPlayer** realizuje cztery fazy MCTS opisane w sekcji 1.2. W turniejach i GUI agenci działają w trybie *online*: przed każdym ruchem wykonywany jest świeży cykl symulacji z zadanym budżetem iteracji, a drzewo nie jest utrzymywane między partiami (**begin_new_game** czyści stan przeszukiwania). Warianty FPU, LGR i PMBp różnią się odpowiednio fazą wyboru, polityką rolloutu oraz sposobem agregacji wyników w propagacji wstecznej.

Metoda **evaluate(state)** zwraca strukturę **SearchEvaluation** z wartością pozycji i oceną poszczególnych ruchów. Służy ona do liczenia metryki *Blunder Rate*: przybliżona wyrocznia

(silnik referencyjny z tagiem **ORACLE**) ocenia pozycje przed ruchem, a *regret* mierzy spadek jakości względem ruchu wskazanego przez tę wyrocznię. Jakość tej metryki zależy wprost od siły i bezstronności wyroczni — w projekcie jest to jedynie pierwsza, ograniczona próba jej zastosowania (szczegóły w sekcji 3.3.2).

2.3.3 Gracz LLM

LLMPlayer komunikuje się z modelem językowym przez interfejs zgodny z API OpenAI (**OpenAIClient**). W każdej turze model reprezentację planszy oraz listę legalnych ruchów, a w pierwszej dodatkowo reguły gry; odpowiedź parsowana jest do obiektu **Move** i walidowana. Przy błędzie parsowania lub nielegalnym ruchu zapytanie jest ponawiane, a po wyczerpaniu prób stosowany jest fallback (losowy legalny ruch). Liczniki żądań i błędów zapisuje turniej do późniejszej analizy.

2.4 Interfejsy użytkownika

CLI umożliwia grę człowiek–agent lub demonstrację agent–agent z linii poleceń, z wyborem typu gracza i podstawowych hiperparametrów. **GUI** (Pygame) oferuje ekran ustawień (tryb jedno- lub dwuosobowy, wybór agenta, konfiguracja iteracji MCTS, głębokość minimaxa, parametry LLM) oraz interaktywną planszę z przełączaniem **drop/push**. Oba interfejsy korzystają z tego samego silnika gry i fabryki agentów.

2.5 Pipeline eksperymentalny

Eksperymenty turniejowe opisuje jeden plik konfiguracyjny w formacie TOML. Globalnie ustawia się m.in. ziarno generatora losowości, katalog wyjściowy i liczbę partii na parę; każdy uczestnik definiowany jest osobnym wpisem z typem oraz hiperparametrami (np. liczba iteracji MCTS, głębokość minimaxa, model LLM). Gracz oznaczony tagiem **ORACLE** służy wyłącznie do oceny blunderów i nie startuje w turnieju round-robin.

Pipeline rozgrywa każdą parę agentów określoną liczbę razy, naprzemiennie zamieniając kolory. Dla każdego ruchu zapisywany jest stan przed posunięciem, co umożliwia późniejszą ocenę wyrocznią. Wyniki agregowane są do tabel podsumowujących turniej. Osobny etap równolegle ocenia unikalne pozycje wyrocznią MCTS; pełny przebieg łączy turniej, analizę i—opcjonalnie—etap *Blunder Rate* w jednym uruchomieniu z możliwością wznowiania.

3 Sprawdzone hipotezy i opis eksperymentów

W niniejszej sekcji zestawiamy hipotezy sformułowane w konspekcie projektu (dostosowane do faktycznie zaimplementowanych reguł gry) oraz opisujemy przeprowadzone eksperymenty: automatyczny turniej agentów, ocenę *Blunder Rate* wyrocznią MCTS oraz testy człowiek–komputer z udziałem pięciu graczy. Szczegóły implementacji agentów i pipeline’u znajdują się w sekcji 2; wyniki liczbowe — w sekcji 4.

3.1 Postawione hipotezy

1. **Supremacja nad ludźmi i LLM.** Warianty MCTS z budżetem 1000 iteracji na ruch (tryb online, bez trwałego drzewa między partiami) osiągają wyższą skuteczność niż lokalnie uruchomione modele językowe oraz niż amatorzy w testach człowiek–komputer, wygrywając z nimi w ponad 75% rozgrywek w warunkach opisanych w sekcji 3.3.
2. **Przewaga UCT nad graczem heurystycznym.** Gracz MCTS w wariantcie UCT z budżetem 1000 iteracji na ruch osiąga wyższy *Win Rate* w turnieju round-robin niż minimax z heurystyką dostosowaną do wariantu suicide (przynajmniej przy głębokości $d = 4$, reprezentującej najmocniejszą wersję heurystyczną w naszym zestawie).

3. **Odporność PMBp na pułapki taktyczne.** Modyfikacja *Power-Mean Backpropagation* z parametrem $p = 2,5$ wykazuje istotnie niższą tendencję do rażących błędów (*Blunder Rate*) niż pozostałe warianty MCTS przy tym samym budżecie iteracji — przy założeniu wiarygodnej wyroczni oceniającej jakość pozycji (hipoteza weryfikowalna tylko w takim stopniu, w jakim spełniony jest ten warunek).
4. **Zbalansowanie rozgrywki.** Plansza 6×8 w połączeniu z regułą natychmiastowej porażki właściciela nowej linii nie wprowadza dominującej przewagi pierwszego gracza: *Win Rate* rozegranych jako czerwony (pierwszy ruch) mieści się w przybliżeniu w przedziale 45%–55% dla silnych agentów, a udział remisów pozostaje istotny zwłaszcza w parach MCTS–MCTS.
5. **Skuteczność modyfikacji UCT.** Co najmniej jedna z badanych modyfikacji UCT (FPU, LGR lub PMBp) osiąga wyższy *Win Rate* w turnieju końcowym niż bazowy UCT przy identycznym budżecie 1000 iteracji na ruch.

Uwaga metodologiczna: w konspekcie rozważano mechanizm *sprawiedliwości turowej*; w ostatecznej implementacji przyjęto regułę **natychmiastowej porażki właściciela linii** (sekcja 1). Hipoteza 4 weryfikowana jest zatem przez analizę *Win Rate* w podziale na kolor startowy oraz *Draw Rate*, a nie przez osobny mechanizm gry.

3.2 Porównywani gracze

Eksperyment końcowy obejmował dziewięciu uczestników turnieju oraz jedną wyrocznię pomocniczą. Tabela 1 zestawia ich typy i hiperparametry. Wszyscy gracze MCTS w turnieju działali w trybie *online*: przed każdym ruchem wykonywano świeże przeszukiwanie z podanym budżetem iteracji, bez utrzymywania drzewa między partiami (por. sekcja 2).

Tabela 1: Skład turnieju końcowego — identyfikatory z konfiguracji TOML.

Identyfikator	Typ	Parametry
uct-1000-1.41	UCT	1000 iter./ruch, $C = 1,41$
fpu-1000-1.0	FPU	1000 iter./ruch, FPU = 1,0
pmbp-1000-2.5	PMBp	1000 iter./ruch, $p = 2,5$
lgr-1000	LGR	1000 iter./ruch
minimax-d4	Minimax	głębokość $d = 4$
minimax-d3	Minimax	głębokość $d = 3$
random	Losowy	—
qwen2.5	LLM	qwen2.5:7b, Ollama, $T=1$
openchat	LLM	openchat:7b, Ollama, $T=1$
uct-20000-1.0-ORACLE	Wyrocznia (UCT)	tag ORACLE, 20 000 iter./pozycja

Gracz oznaczony tagiem ORACLE nie brał udziału w turnieju round-robin — służył wyłącznie do oceny jakości ruchów (*Blunder Rate*). Modele językowe komunikowały się z lokalnym serwerem Ollama (interfejs zgodny z API OpenAI); w turnieju automatycznym uruchamiano je bez dodatkowego *chain-of-thought*, z walidacją i ponawianiem nielegalnych odpowiedzi (szczegóły w sekcji 2).

3.3 Metodyka eksperymentalna

3.3.1 Turniej automatyczny round-robin

Główny eksperyment przeprowadzono z użyciem opisanej wyżej konfiguracji turniejowej. Dla każdej nieuporządkowanej pary graczy rozegrano **10 partii**, przy naprzemiennym przypisywaniu kolorów (pięć partii jako czerwony, pięć jako żółty). Łącznie wykonano $\binom{9}{2} \times 10 = 360$ gier.

Ziarno generatora losowości ustawiono na 0; dla partii i w parze (A, B) stosowano deterministyczne ziarno wyliczane z indeksu pary i numeru gry, co umożliwia odtworzenie przebiegu turnieju. Dla każdej partii zapisywano m.in. identyfikator pary, wynik, liczbę ruchów, czasy decyzji obu agentów oraz — w przypadku LLM — liczbę zapytań i błędów parsowania. Pełny stan przed każdym ruchem stanowi podstawę późniejszej oceny wyroczni.

3.3.2 Wyrocznia i metryka Blunder Rate

W projekcie przyjęto **przybliżoną** wyrocznię do oceny jakości ruchów — nie jest to silnik uznawany za optymalny w sensie teoretycznym, lecz pierwsza praktyczna próba zastosowania metryki *Blunder Rate* w tym wariancie gry. Świadome ograniczenia tej próby należy brać pod uwagę przy interpretacji hipotezy 3 i wszystkich wyników dotyczących blunderów.

Konstrukcja wyroczni. Silnik referencyjny to zwykły UCT (bez FPU/LGR/PMBp). Przy ocenie pozycji z turnieju stosowano 20 000 iteracji na stan — więcej niż agenci turniejowi (1000/ruch), lecz nadal skończony budżet Monte Carlo.

Definicja blundera. Ruch uznaje się za **blunder**, jeśli wartość pozycji po wybranym ruchu (w skali $[0, 1]$) jest o więcej niż $\tau = 0,3$ niższa od wartości ruchu wskazanego przez wyrocznię. *Blunder Rate* to odsetek takich ruchów; *regret* to średnia różnica wartości między ruchem wyroczni a ruchem agenta.

Ułomności pomiaru.

- Wyrocznia sama jest UCT-em — może *systematycznie faworyzować* ocenę ruchów podobnych do własnej polityki i niedoszacowywać wartość strategii charakterystycznych dla PMBp, FPU lub LGR.
- 20 000 iteracji to kompromis między czasem obliczeń a stabilnością oceny; nie ma gwarancji zbieżności wartości korzenia do „prawdziwej” jakości pozycji w tak złożonym wariancie.
- Drzewo wyroczni budowane było w self-playu UCT-a i może *slabo pokrywać* pozycje osiąganę przez innych agentów w turnieju.
- Próg $\tau = 0,3$ jest arbitralny; przy silnych agentach MCTS odsetek blunderów spada do ok. 0,1–0,2%, co daje **liczbę pojedynczych zdarzeń** (1–9 na tysiąc ruchów) — zbyt mało, by wiarygodnie rozróżniać warianty MCTS bez znacznie większej próby i lepszej wyroczni.

Z powyższych powodów wyniki *Blunder Rate* traktujemy jako **materiał pomocniczy i eksploracyjny**, a nie jako rozstrzygające dowody na korzyść lub niekorzyść któregoś z wariantów MCTS w sensie hipotezy 3.

3.3.3 Porównanie z modelami językowymi

W turnieju automatycznym uczestniczyły dwa modele uruchomione lokalnie: `qwen2.5:7b` oraz `openchat:7b`. Każdy model rozegrał po 80 partii (po 10 z każdym z pozostałych ośmiu przeciwników). Zebrano metryki niezawodności interfejsu (odsetek niepoprawnych odpowiedzi, udział ruchów losowych po wyczerpaniu ponowień).

3.4 Testy człowiek–komputer

Uzupełnieniem turnieju automatycznego były **testy z udziałem pięciu graczy ludzkich**. Każda osoba rozgrywała partie przeciwko wybranym agentom za pośrednictwem graficznego interfejsu użytkownika (Pygame). Uczestnik wybierał kolor (**drop/push**, kolumna) zgodnie z tymi

samymi regułami wariantu suicide co w turnieju maszynowym; przeciwnik był konfigurowany z poziomu GUI (typ agenta oraz hiperparametry, np. liczba iteracji MCTS lub głębokość minimaxa).

Protokół testów:

- **Liczba uczestników:** 5 absolwentów kierunków technicznych z tytułem inżyniera bez wcześniejszego treningu w tym wariantcie gry (poza krótkim wprowadzeniem do zasad).
- **Przeciwnicy:** siedmiu agentów z turnieju końcowego — `random`, `minimax` (`minimax-d3`, `minimax-d4`) oraz cztery warianty MCTS (`uct-1000-1.41`, `fpu-1000-1.0`, `pmbp-1000-2.5`, `lgr-1000`) z tymi samymi hiperparametrami co w turnieju automatycznym.
- **Organizacja partii:** każdy uczestnik rozgrywał **dwie** partie z każdym agentem — raz jako czerwony, raz jako żółty — łącznie $5 \times 7 \times 2 = 70$ gier.
- **Cel:** weryfikacja hipotezy 1 (supremacja MCTS nad człowiekiem) oraz jakościowe porównanie trudności gry dla człowieka z wynikami turnieju agentów.

3.5 Metryki oceny

Do weryfikacji hipotez wykorzystano następujące metryki (zgodnie z metodologią z konspektu, w zakresie faktycznie zaimplementowanym w pipeline’ie):

- **Win Rate** — odsetek partii zakończonych zwycięstwem danego gracza; liczony łącznie oraz osobno dla gier jako czerwony i jako żółty (*side balance*).
- **Draw Rate** — odsetek remisów; istotny przy ocenie zbalansowania wariantu (hipoteza 4) oraz strategii defensywnych.
- **Średnia długość partii** — liczba ruchów do zakończenia gry; krótkie partie często wskazują na szybkie wykorzystanie pułapki taktycznej lub rażący błąd.
- **Czas decyzji** — średni czas wyboru ruchu [s]; porównanie kosztu obliczeniowego MCTS, minimaxa i LLM.
- **Blunder Rate** — odsetek ruchów z $\text{regret} > \tau = 0,3$ względem przybliżonej wyroczni UCT (20 000 iteracji); metryka pomocnicza przy weryfikacji hipotezy 3, o ograniczonej mocy rozróżniającej (sekcja 3.3.2).
- **Średni regret** — średnia strata wartości pozycji względem ruchu wskazanego przez wyrocznię; podlega tym samym zastrzeżeniom co *Blunder Rate*.
- **Metryki LLM** — odsetek niepoprawnych odpowiedzi modelu i udział ruchów losowych po błędzie.

Tabela 2 wiąże hipotezy z eksperymentami i metrykami weryfikacji.

Tabela 2: Mapowanie hipotez na eksperymenty i metryki.

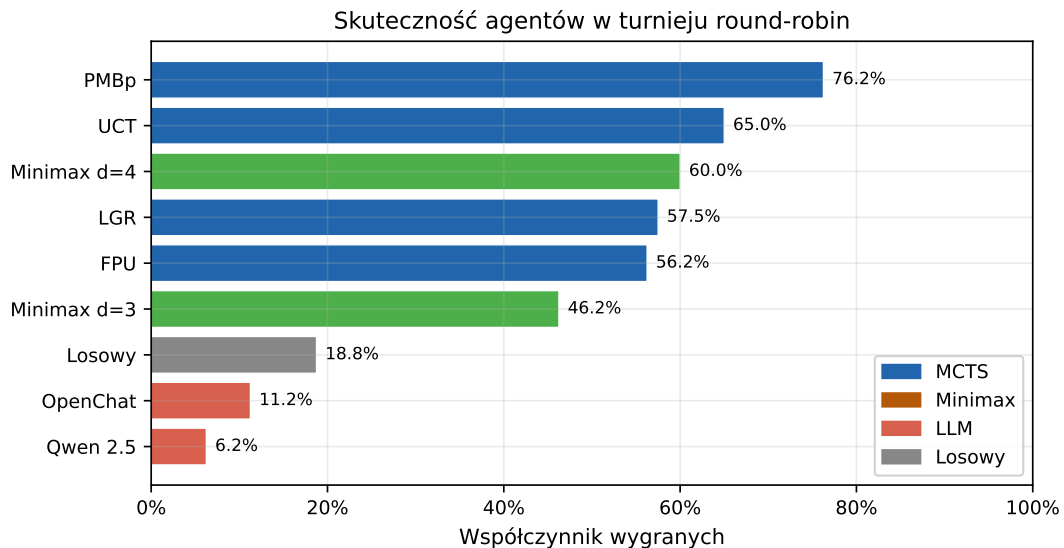
Hipoteza	Eksperyment	Metryki
1. Supremacja nad ludźmi i LLM	Turniej automatyczny + testy człowiek–komputer	Win Rate agenta > 75% <i>vs</i> LLM i człowiek
2. UCT <i>vs</i> minimax	Pary UCT–minimax w turnieju	Win Rate, bezpośrednie pojedynki
3. Odporność PMBp	Ocena <i>Blunder Rate</i> (wyrocznia UCT)	Blunder Rate — przy zastrzeżeniach sekcji 3.3.2
4. Zbalansowanie	Wszystkie pary turnieju + testy człowiek–komputer	Win Rate wg koloru, Draw Rate
5. Modyfikacje UCT	Ranking turniejowy, pary MCTS	Win Rate PMBp/FPU/LGR <i>vs</i> UCT

4 Wyniki eksperymentów

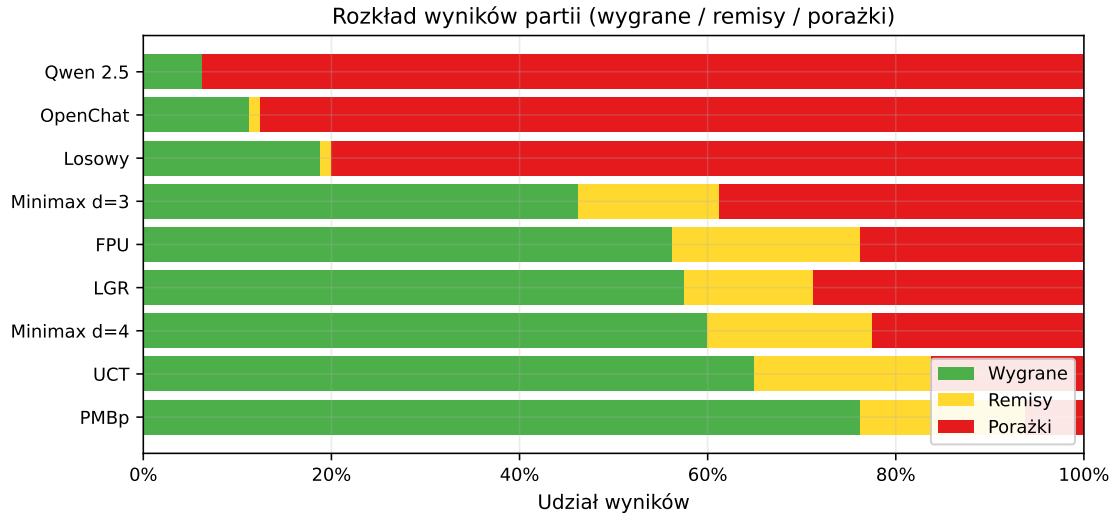
Poniżej przedstawiamy wyniki eksperymentu końcowego: turnieju round-robin dziewięciu agentów (360 partii), wstępnej oceny *Blunder Rate* na ok. 9400 ruchach (z zastrzeżeniami co do wyroczni, sekcja 3.3.2) oraz testów z pięcioma graczami ludzkimi i porównania z modelami językowymi.

4.1 Wyniki turnieju agentów

Turniej round-robin rozegrano w układzie dziewięciu graczy, po 80 partii na agenta. Ranking według *Win Rate* przedstawia tabela 3 i rysunek 2. Pierwsze trzy miejsca zajęli PMBp (76,3%, 61 wygranych na 80), UCT (65,0%) oraz minimax $d=4$ (60,0%); kolejne pozycje: LGR (57,5%), FPU (56,3%), minimax $d=3$ (46,3%), gracz losowy (18,8%), OpenChat (11,3%), Qwen 2.5 (6,3%).



Rysunek 2: Skuteczność agentów w turnieju round-robin (80 partii na gracza).



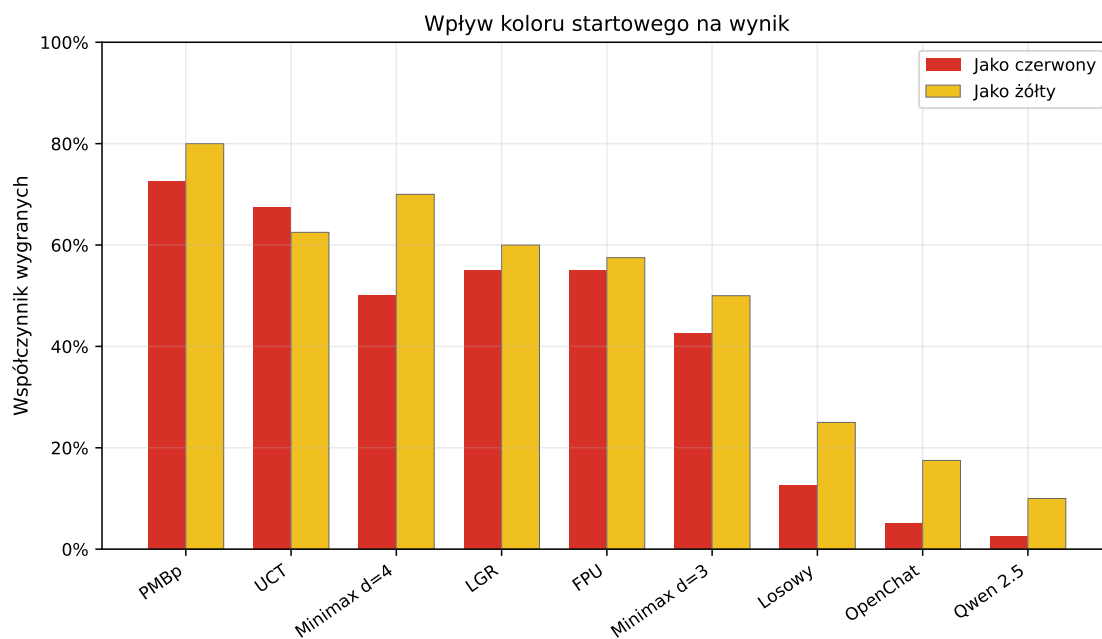
Rysunek 3: Rozkład wyników: wygrane, remisy i porażki według agenta.

Tabela 3: Podsumowanie turnieju końcowego.

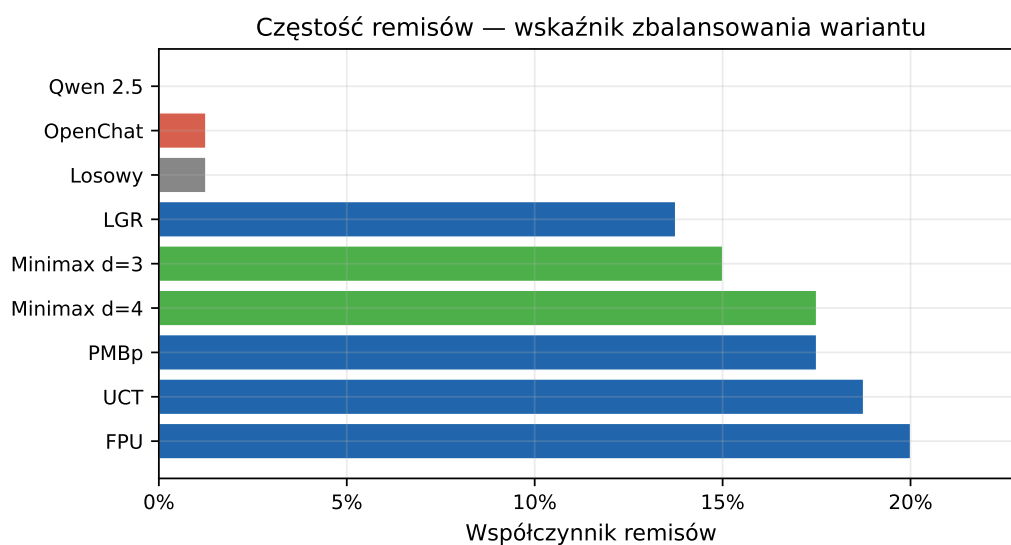
Gracz	M.	Win rate	Draw rate	Śr. ruchów	Śr. czas [s]
PMBp	1	76,3%	17,5%	30,4	1,28
UCT	2	65,0%	18,8%	30,2	1,26
Minimax $d=4$	3	60,0%	17,5%	31,7	0,63
LGR	4	57,5%	13,8%	31,1	0,96
FPU	5	56,3%	20,0%	29,9	1,51
Minimax $d=3$	6	46,3%	15,0%	31,6	0,13
Losowy	7	18,8%	1,3%	18,4	≈ 0
OpenChat	8	11,3%	1,3%	18,2	0,65
Qwen 2.5	9	6,3%	0,0%	13,3	0,56

Średnia długość partii wynosiła ok. 30–32 ruchy u agentów MCTS i minimaxa oraz ok. 13–18 ruchów w grach z udziałem LLM lub gracza losowego. W turnieju PMBp zanotował 61 zwycięstw, 14 remisów i 5 porażek na 80 partii.

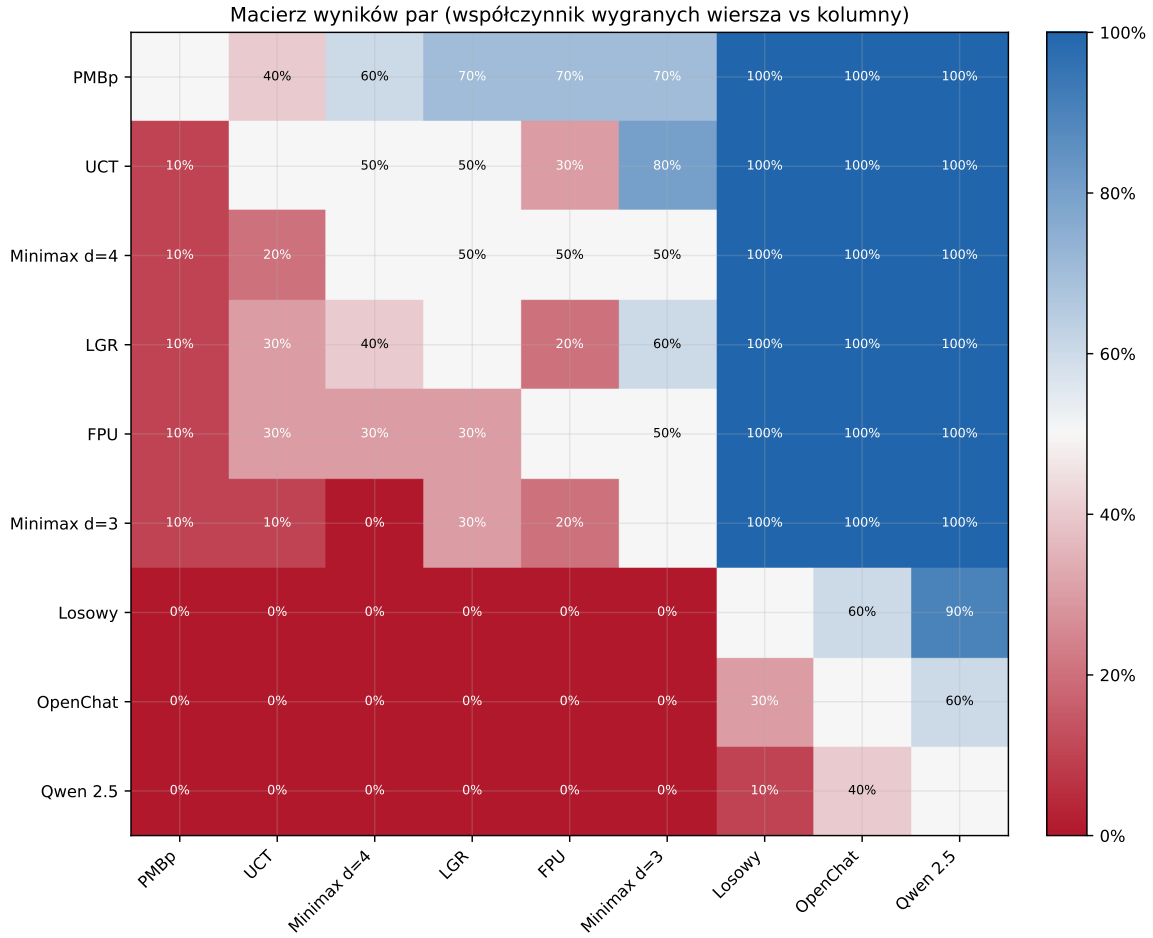
Zbalansowanie kolorów startowych. Rysunek 4 zestawia *Win Rate* w podziale na kolor startowy (czerwony = pierwszy ruch). Dla PMBp wartości wynoszą 72,5% (czerwony) i 80,0% (żółty); dla UCT — 67,5% i 62,5%. U części pozostałych agentów oba wskaźniki są bliższe siebie.



Rysunek 4: Współczynnik wygranych w zależności od koloru startowego (czerwony = pierwszy ruch).



Rysunek 5: Częstość remisów według agenta.



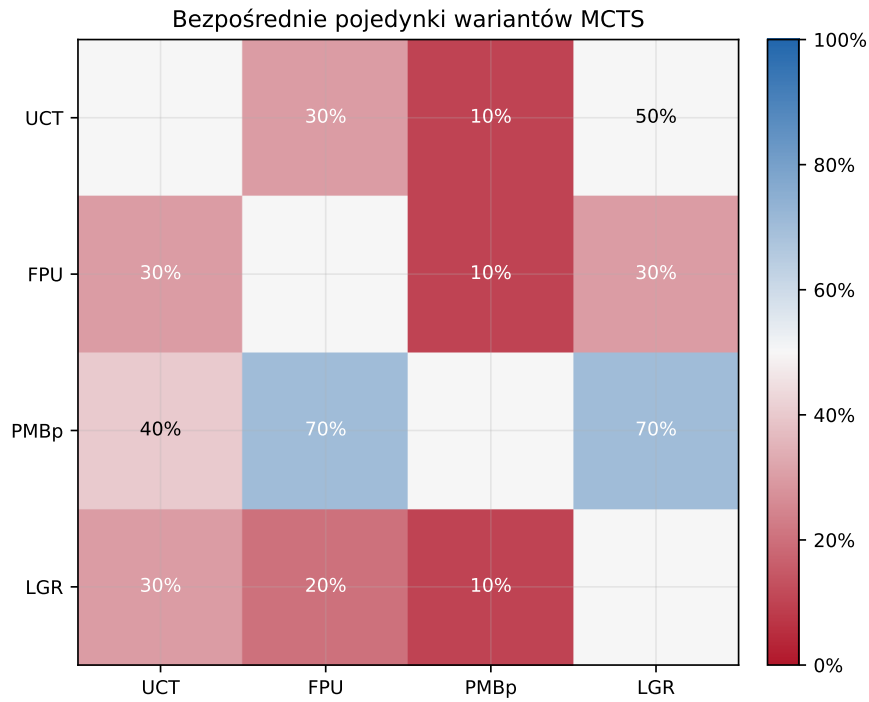
Rysunek 6: Macierz wyników par: wartość w komórce (i, j) to *Win Rate* gracza wiersza przeciwko graczowi kolumny.

4.2 Porównanie wariantów MCTS

Cztery warianty MCTS miały identyczny budżet 1000 iteracji na ruch. Tabela 4 zestawia bezpośrednie pojedynki; rysunek 7 pokazuje je w formie macierzy.

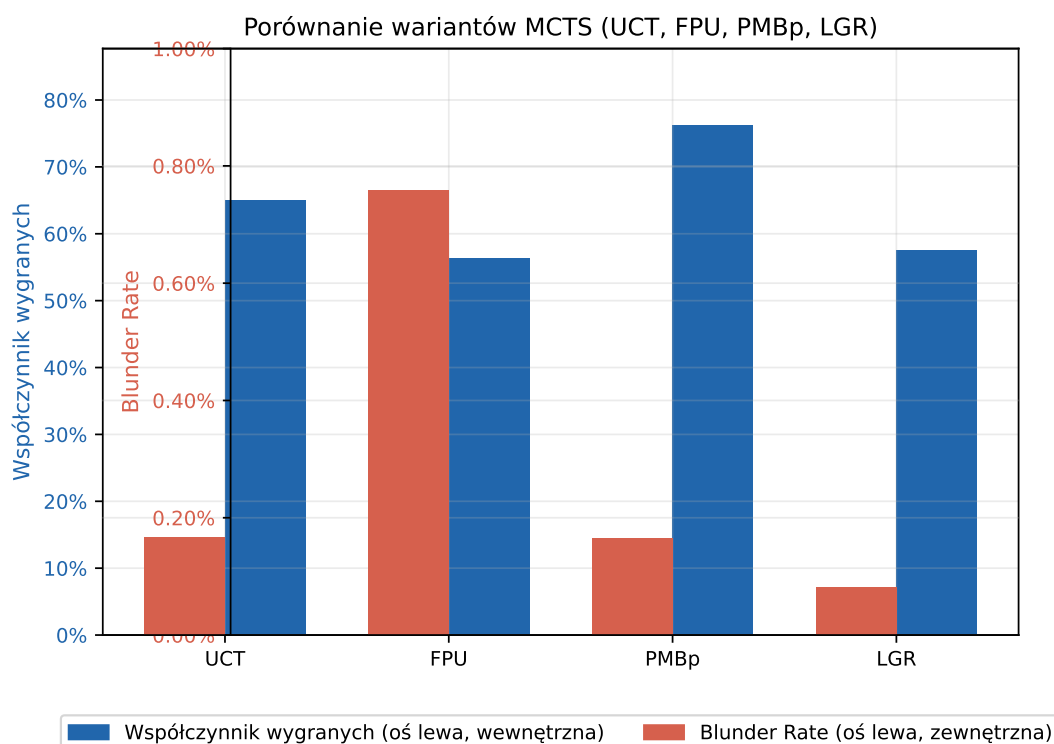
Tabela 4: Wybrane pary bezpośrednie między wariantami MCTS (format: wygrane–przegrane–remisy przy 10 grach).

Para	Wynik	Draw rate
PMBp vs LGR	7–1–2	20%
PMBp vs FPU	7–1–2	20%
PMBp vs UCT	4–1–5	50%
UCT vs LGR	5–3–2	20%
UCT vs FPU	3–3–4	40%
FPU vs LGR	3–2–5	50%



Rysunek 7: Bezpośrednie pojedynki UCT, FPU, PMBp i LGR.

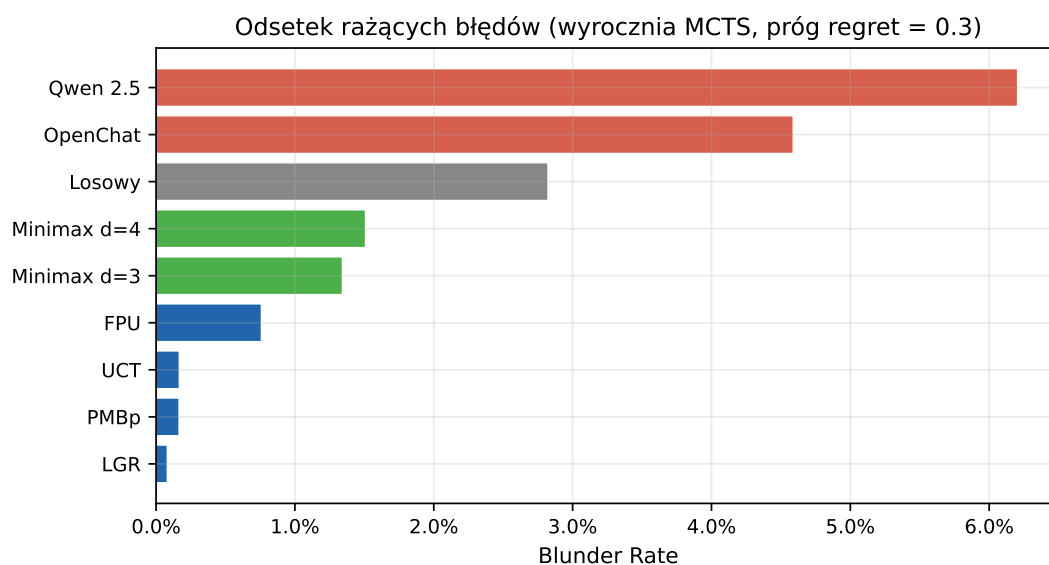
W parach między wariantami MCTS (po 10 gier na parę) PMBp uzyskał łącznie 18 zwycięstw, 3 porażki i 9 remisów. W bezpośrednim pojedynku PMBp–UCT wynik wyniósł 4–1–5, PMBp–FPU 7–1–2, PMBp–LGR 7–1–2, UCT–LGR 5–3–2, UCT–FPU 3–3–4, FPU–LGR 3–2–5. W parach UCT–minimax $d=3$ i UCT–minimax $d=4$ odpowiednio 8–1–1 oraz 5–2–3; w parach PMBp–minimax $d=3$ i PMBp–minimax $d=4$ — 7–1–2 oraz 6–1–3. Średni czas decyzji minimaxa wynosił 0,13–0,63 s na ruch, MCTS ok. 1,3 s.



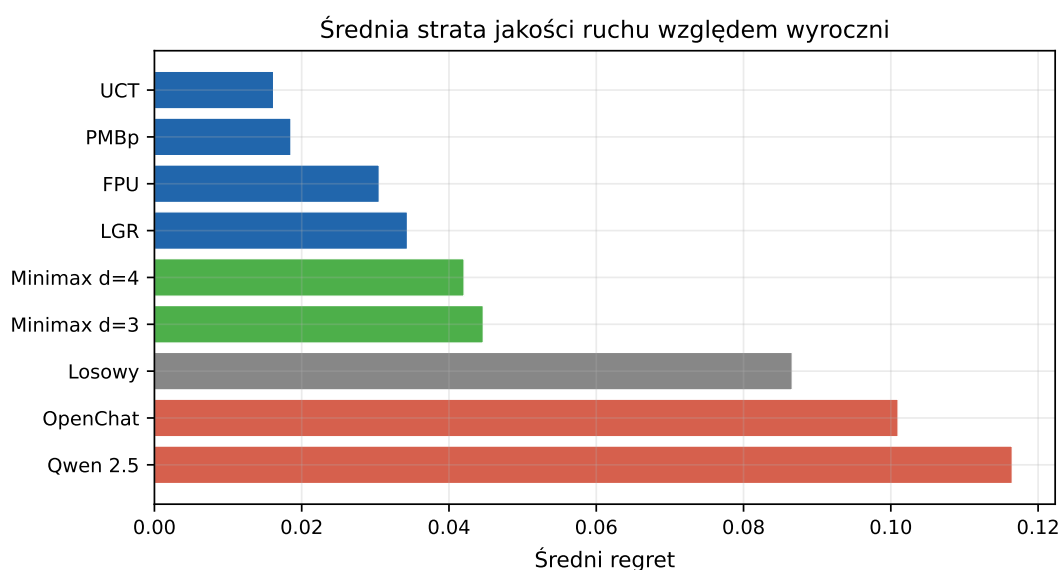
Rysunek 8: Porównanie wariantów MCTS: *Win Rate* i orientacyjny *Blunder Rate* na osobnych skalach osi *y* po lewej stronie (patrz sekcja 3.3.2).

4.3 Blunder Rate i jakość gry

Ocena wyrocznią (sekcja 3.3.2) objęła wszystkie ruchy agentów turniejowych. Wśród wariantów MCTS odsetek blunderów wyniósł: LGR 0,08% (1/1239), PMBp i UCT po 0,17% (2/1210 i 2/1198), FPU 0,76% (9/1187). Minimax $d=3$ i $d=4$ osiągnął ok. 1,3% i 1,5%; gracz losowy 2,8%; modele LLM 4,6–6,2%.

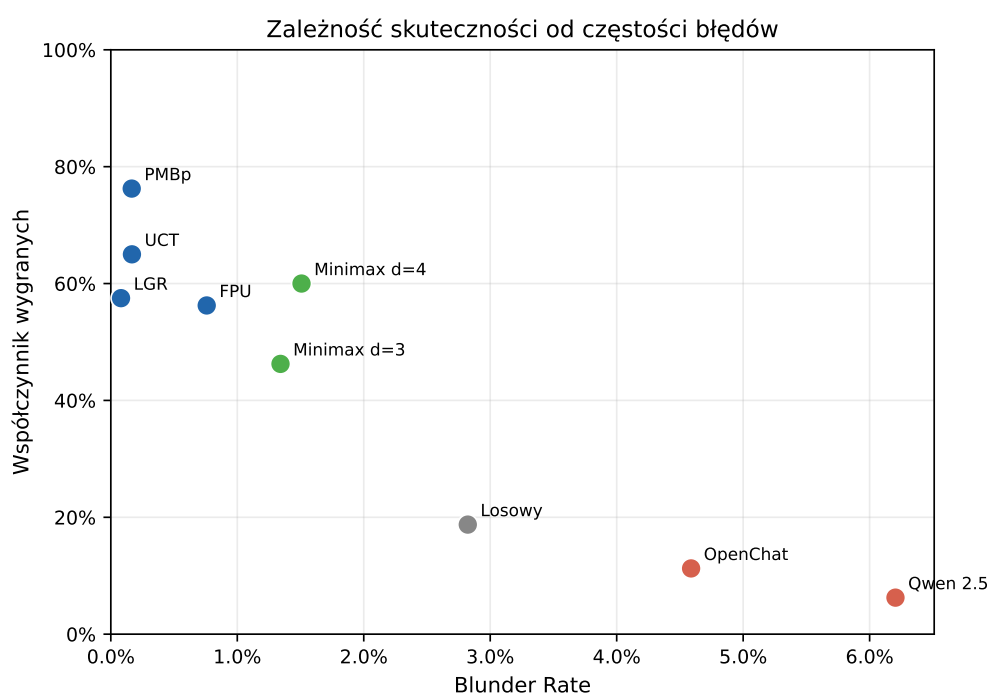


Rysunek 9: Orientacyjny *Blunder Rate* według agenta ($\tau = 0,3$, wyrocznia UCT 20 000 iteracji; patrz sekcja 3.3.2).



Rysunek 10: Średni regret względem ruchu wskazanego przez wyrocznię (patrz sekcja 3.3.2).

Średni regret (rysunek 10) jest najwyższy u LLM i gracza losowego, niższy u wariantów MCTS. Rysunek 11 zestawia *Win Rate* z orientacyjnym *Blunder Rate* dla każdego agenta turniejowego; FPU ma 56,3% wygranych przy wyższym odsetku blunderów wg wyroczni niż pozostałe warianty MCTS.



Rysunek 11: Zależność *Win Rate* od orientacyjnego *Blunder Rate* (każdy punkt to jeden agent turniejowy).

4.4 Testy z udziałem człowieka i modeli językowych

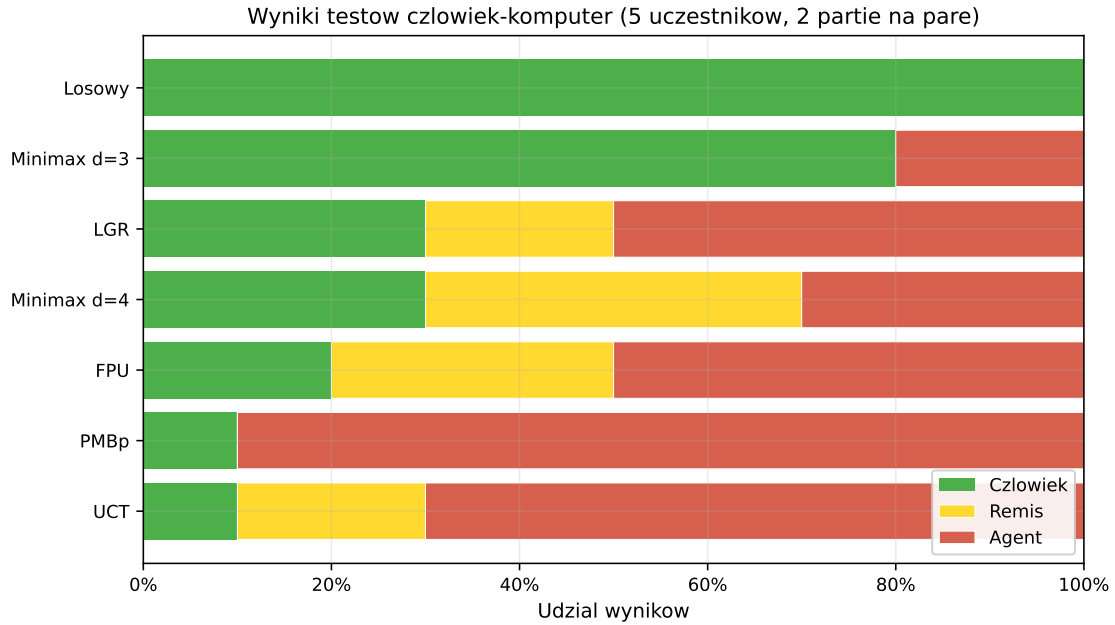
4.4.1 Testy człowiek–komputer

Pięciu uczestników rozegrało łącznie 70 partii przeciwko siedmiu agentom (po dwie gry na parę, z naprzemiennym kolorem). Tabela 5 zestawia współczynnik wygranych człowieka i agenta;

rysunek 12 przedstawia te same dane graficznie.

Tabela 5: Wyniki testów człowiek–komputer (10 partii na agenta, łącznie 70 gier).

Agent	Wygrane człowieka	Wygrane agenta	Remisy	Win rate człowieka
Losowy	10	0	0	100%
Minimax $d=3$	8	2	0	80%
Minimax $d=4$	3	3	4	30%
LGR	3	5	2	30%
FPU	2	5	3	20%
UCT	1	7	2	10%
PMBp	1	9	0	10%
Łącznie	28	31	11	40,0%
MCTS (razem)	7	26	7	17,5%

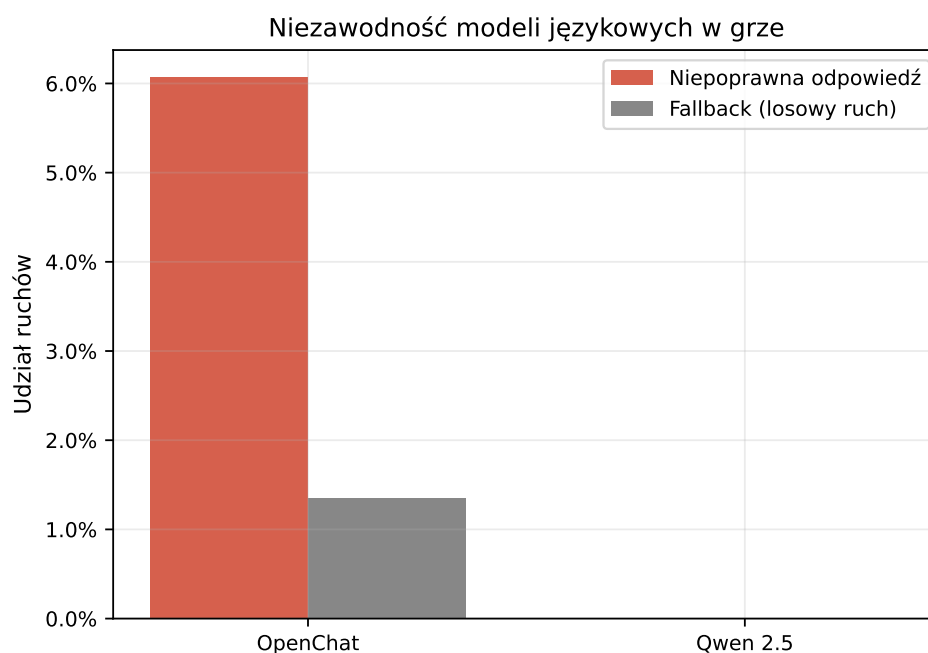


Rysunek 12: Porównanie skuteczności człowieka i agenta w testach jeden na jeden (5 uczestników $\times$ 2 partie na parę; procent wygranych liczony wśród partii zakończonych zwycięstwem, bez remisów w mianowniku).

Łączny *Win Rate* człowieka jako czerwony wyniósł 31,4% (11/35), jako żółty — 48,6% (17/35). Przeciwno wariantom MCTS uczestnicy wygrali 7 partii, przegrali 26 i zremisowali 7; przeciwko graczowi losowemu — 10/10; przeciwko minimaxowi $d=3$ — 8/10; przeciwko minimaxowi $d=4$ — 3 wygrane, 3 porażki i 4 remisy.

4.4.2 Porównanie między modelami językowymi

W turnieju automatycznym modele `qwen2.5:7b` i `openchat:7b` osiągnęły odpowiednio 6,3% i 11,3% *Win Rate* (po 80 partiach każdy). Gracz losowy miał 18,8%. Średnia długość partii LLM wynosiła 13,3–18,2 ruchów. W bezpośrednich parach z agentami turniejowymi modele przegrywały większość gier; `openchat:7b` wygrał 3 z 10 partii z graczem losowym.

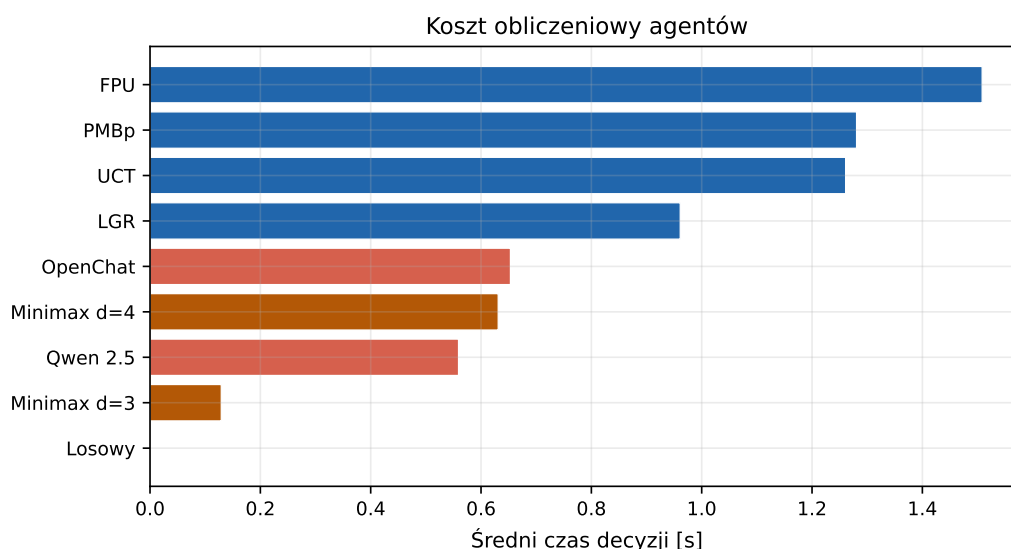


Rysunek 13: Niezawodność odpowiedzi modeli językowych: odsetek niepoprawnych odpowiedzi API oraz ruchów losowych po błędzie (fallback).

Metryki niezawodności interfejsu (rysunek 13): OpenChat zwrócił niepoprawną odpowiedź w 6% zapytań; Qwen 2.5 nie zanotował błędów parsowania i przegrał 75 z 80 partii w turnieju.

4.5 Obserwacje dodatkowe

- **Remisy.** W parach MCTS–MCTS *Draw Rate* w pojedynczych seriach sięgał 40–50% (np. UCT–PMBp 5 remisów na 10 gier, UCT–FPU 4 na 10).
- **Czas decyzji.** FPU: ok. 1,51 s/ruch; UCT i PMBp: ok. 1,26–1,28 s; minimax $d=3$: 0,13 s; minimax $d=4$: 0,63 s (tabela 3, rysunek 14).
- **Tryb MCTS.** Agenci turniejowi nie utrzymywali drzewa między partiami; przed każdym ruchem wykonywano świeże przeszukiwanie z budżetem 1000 iteracji.



Rysunek 14: Średni czas decyzji agentów w turnieju automatycznym.

5 Wnioski

Poniżej podsumowujemy weryfikację hipotez z sekcji 3.1, z wyraźnym rozróżnieniem między tym, co wynika z danych, a tym, co pozostaje otwarte ze względu na małą liczbę partii i ograniczenia metodologiczne.

5.1 Weryfikacja hipotez

1. **Supremacja nad ludźmi i LLM — częściowo potwierdzona.** *Wobec LLM:* w turnieju każdy z czterech wariantów MCTS wygrywał z modelami językowymi w ponad 75% partii bezpośrednich; OpenChat i Qwen 2.5 osiągnęły odpowiednio 11,3% i 6,3% *Win Rate*. Próg 75% jest spełniony dla MCTS *vs* LLM w badanych warunkach (lokalne modele 7B, temperatura 1).

Wobec człowieka: próg 75% zwycięstw MCTS *nie* został spełniony — w 40 partiach przeciwko czterem wariantom MCTS agenci wygrali 26 razy, uczestnicy 7 i zremisowano 7 (65% wygranych MCTS w liczbie wszystkich partii). W 70 partiach łącznie ludzie wygrali 28 razy, przegrali 31 i zremisowali 11. Najwięcej wygranych człowieka przeciwko LGR (3/10) i FPU (2/10), najmniej przeciwko PMBp i UCT (po 1/10). Ci sami uczestnicy pokonali gracza losowego (10/10) i minimax $d=3$ (8/10); z minimaxem $d=4$ wynik to 3–3–4. Ogólna „supremacja AI nad człowiekiem” *nie* została wykazana. Próba jest mała ($n=5$, 10 partii na agenta) i nie obejmuje graczy doświadczonych.

Uwaga: hipoteza odnosiła się do MCTS w trybie online (1000 iteracji/ruch), a nie do agentów z długim treningiem self-play; wyniki nie należy utożsamiać z „wytrenowanym” silnikiem szachowym.

2. **Przewaga UCT nad graczem heurystycznym — częściowo potwierdzona.** W turnieju round-robin UCT (65,0%) uzyskał wyższy *Win Rate* niż minimax $d=4$ (60,0%) i wyraźnie wyższy niż $d=3$ (46,3%). W bezpośrednich parach: UCT–minimax $d=4$ to 5–2–3, UCT–minimax $d=3$ to 8–1–1. Różnica względem $d=4$ jest jednak umiarkowana (trzy remisy na dziesięć gier), a minimax $d=4$ wciąż plasuje się na trzecim miejscu w całym turnieju — przed częścią wariantów MCTS. Hipotezę należy uznać za potwierdzoną względem słabszego minimaxa i prawdopodobną względem $d=4$, ale nie jako rozstrzygającą przewagę nad dobrze dobraną heurystyką.

3. **Odporność PMBp na pułapki taktyczne — nierozstrzygnięta (brak wiarygodnych danych).** Hipoteza wymaga porównania *Blunder Rate* PMBp z innymi wariantami MCTS przy założeniu trafnej wyroczni. W eksperymencie wyrocznią był UCT z 20 000 iteracji na pozycję (sekcja 3.3.2) — rozwiązanie celowo uproszczone i prawdopodobnie *stronnicze* wobec polityki UCT-a. Zaobserwowane wartości u silnych MCTS (0,08–0,76%) odpowiadają jedynie kilku zdarzeniom na tysiąc ruchów; różnica między LGR (1 blunder) a PMBp i UCT (po 2) nie pozwala na żadne rzetelne wnioskowanie. FPU ma wyższy wskaźnik (9 blunderów), lecz bez niezależnej wyroczni nie wiadomo, czy jest to artefakt pomiaru.

Nie uznajemy hipotezy za obaloną: brak dowodu na wyższą odporność PMBp nie jest dowodem na jej brak — po prostu **nie zebrano danych** o jakości wymaganej do weryfikacji. Rozstrzygnięcie wymagałoby m.in. silniejszej lub bardziej bezstronnej wyroczni (np. głębszy minimax, większy budżet MCTS, analiza ludzka), większej liczby pozycji oraz ewentualnie niższego progu τ lub innych metryk jakości ruchu.

4. **Zbalansowanie rozgrywki — niejednoznaczna / słabo potwierdzona.** Wysoki *Draw Rate* u silnych MCTS (do ok. 20%, w parach MCTS–MCTS nawet 40–50%) wskazuje na zacięty, defensywny charakter wariantu. Jednak kryterium z hipotezy — *Win Rate* gracza rozpoczynającego w przedziale 45–55% — nie jest spełnione u najlepszych agentów (PMBp: 72,5% jako czerwony; UCT: 67,5%). W testach ludzkich *Win Rate* jako czerwony wyniósł 31,4% (11/35), jako żółty — 48,6% (17/35). Plansza 6×8 łagodzi przewagę pierwszego ruchu względem klasycznego 6×7 w sposób jakościowy, lecz nie można twierdzić o pełnym zbalansowaniu w sensie liczbowym przyjętym w hipotezie.
5. **Skuteczność modyfikacji UCT — potwierdzona.** PMBp osiągnął 76,3% *Win Rate*, wyższy niż bazowy UCT (65,0%), i wygrał bezpośrednie pojedynki z UCT (4–1–5), FPU (7–1–2) oraz LGR (7–1–2). FPU (56,3%) i LGR (57,5%) *nie* przewyższyły UCT w rankingu ogólnym. Warunek hipotezy („co najmniej jedna modyfikacja”) jest spełniony przez PMBp; pozostałe modyfikacje w badanej konfiguracji nie poprawiły wyniku względem UCT.

5.2 Podsumowanie

Zaimplementowano pełny pipeline do gry w wariant *Connect4* typu suicide (6×8 , ruchy drop/push) z agentami MCTS (UCT, FPU, LGR, PMBp), minimaxem, graczem losowym i LLM, oraz wstępną metryką *Blunder Rate* opartą o przybliżoną wyrocznię UCT. Eksperyment końcowy (360 partii turniejowych, ocena ok. 9400 ruchów, 70 gier człowiek–komputer) pozwala sformułować następujące wnioski ostrożne:

- W badanym wariantcie **PMBp** z $p = 2,5$ okazał się najskuteczniejszym wariantem MCTS przy budżecie 1000 iteracji/ruch, wyprzedzając bazowy UCT zarówno w rankingu round-robin, jak i w bezpośrednich parach.
- **MCTS w trybie online** wyraźnie przewyższa lokalne modele LLM w turnieju; w testach z człowiekiem wygrywa w większości partii przeciwko MCTS (26/40), lecz uczestnicy zdobyli też 7 zwycięstw i 7 remisów.
- **Blunder Rate** w obecnej postaci (wyrocznia UCT, $\tau = 0,3$) z grubsza odróżnia słabych graczy od MCTS, lecz **nie pozwala** zweryfikować hipotezy o taktycznej przewadze PMBp — metryka wymaga dalszego dopracowania, zanim stanie się narzędziem rozstrzygającym.
- **FPU** w tej konfiguracji ($FPU = 1,0$) nie poprawił wyniku względem UCT w turnieju; wyższy *Blunder Rate* wg wyroczni UCT-a nie jest wystarczającym dowodem gorszej gry taktycznej.

Ograniczenia. Po 10 partiach na parę wyniki turniejowe mają ograniczoną moc statystyczną; remisy i pojedyncze wyniki par mogłyby się zmienić przy dłuższym turnieju. Wyrocznia do *Blunder Rate* jest świadomie przybliżona (UCT, skończony budżet, możliwa stronniczość) i nie zastępuje silniejszego silnika referencyjnego — metryka nie rozstrzyga jakości ruchów między silnymi wariantami MCTS (sekcja 3.3.2). Testy ludzkie ($n = 5$) przeprowadzono przy braku trzeźwości uczestników i bez kontroli poziomu umiejętności. Modele LLM (7B, lokalnie) nie uogólniają się na populację graczy ani na modele językowe. MCTS w turnieju nie był trenowany self-playem przed rozgrywką.

Kierunki dalszych prac obejmowałyby zwiększenie liczby partii i liczby ziaren losowości, trening trwałego drzewa przed turniejem, głębszy minimax lub uczenie funkcji oceny, powtórzenie testów ludzkich i z LLM na większej próbie i silniejszych modelach, oraz — w kontekście hipotezy 3 — zbudowanie **silniejszej i bardziej bezstronnej wyroczni** do *Blunder Rate* (większy budżet, niezależna od ocenianego wariantu MCTS, ewentualnie weryfikacja na podzbiórze pozycji przez człowieka).

6 Deklaracja użycia narzędzi sztucznej inteligencji

W ramach projektu modele językowe zostały wykorzystane przy:

- Tworzeniu GUI aplikacji
- Tworzeniu kodu do obsługi gry z LLMami
- Tworzeniu skryptów przeprowadzających eksperymenty
- Tworzenia wykresów i tabel z wynikami
- Wygenerowania bibliografii

Literatura

- [1] Victor Allis. A knowledge-based approach of connect-four: The game is solved: White wins. *ICGA Journal*, 11(4):165–191, 1988.
- [2] Cameron B. Browne, Edward Powley, Daniel Whitehouse, Simon M. Lucas, Peter I. Cowling, Philipp Rohlfshagen, Stephen Tavener, Diego Perez, Spyridon Samothrakis, and Simon Colton. A survey of monte carlo tree search methods. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(1):1–43, 2012.
- [3] Tristan Cazenave. Batch monte carlo tree search. In *International Conference on the Applications of Evolutionary Computation*, pages 3–18. Springer, 2021.
- [4] Tuan Dam, Pascal Klink, Carlo D’Eramo, Jan Peters, and Joni Pajarinen. Generalized mean estimation in monte-carlo tree search. In *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence*, pages 2633–2639, 2020.
- [5] Peter D. Drake. The last-good-reply policy for monte-carlo go. *ICGA Journal*, 32(4):221–227, 2009.
- [6] Levente Kocsis and Csaba Szepesvári. Bandit based monte-carlo planning. In *European Conference on Machine Learning*, pages 282–293. Springer, 2006.
- [7] Stephan Schiffel and Michael Thielscher. Evaluation functions in general game playing. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2010.
- [8] Kavita Sheoran, Geetika Dhand, Mayank Dabas, Nishthavan Dahiya, and Pratish Pushparaj. Solving connect 4 using optimized minimax and monte carlo tree search. *Advances and Applications in Mathematical Sciences*, 21(6):3303–3313, 2022.

6.1 Uwaga pokóńcowa

W wieczór przed terminem oddania projektu, została znaleziona strategia wygrywająca dla gracza żółtego. Celem dalszych badań należałoby rozważyć wprowadzenie jakiegoś mechanizmu sprawiedliwości turowej, np. grania zawsze do wypełnienia całej planszy i zliczania linii.